

Лабораторная работа № 2

«Поток в полости при движении верхней грани»

Аннотация

В этом уроке рассмотрены этапы пре-процессинга, расчета и постпроцессной обработки примера, рассматривающего изотермическое течение несжимаемой жидкости в двумерной квадратной области. Изначально поток будет предполагаться ламинарным и будет решаться на однородной сетке с использованием решателя isoFoam (решатель для ламинарного, изотермического, несжимаемого потока). В ходе урока будет исследовано влияние шага сетки на результаты расчетов. В конце урока будет увеличено число Рейнольдса, следовательно, появится необходимость использовать решатель для турбулентного, изотермического, несжимаемого потока – pisoFoam.

План занятия

- пре-процессная подготовка;
- создание сетки;
- постобработка;
- увеличение разрешения сетки;
- настройка сетки;
- увеличение числа Рейнольдса;
- моделирование потока с высоким числом Рейнольдса;
- изменение геометрии;
- постобработка измененной геометрии.

Форма представления выполненных заданий

Результаты расчетов, полученные в рамках данной лабораторной работы, прикладываются в качестве ответа в том же разделе LMS.

1 Пре-процессная подготовка

Все границы геометрии представляют из себя твердые стенки. Верхняя грань движется в направлении оси x со скоростью 1 м/с, остальные три неподвижны (Рисунок 1).

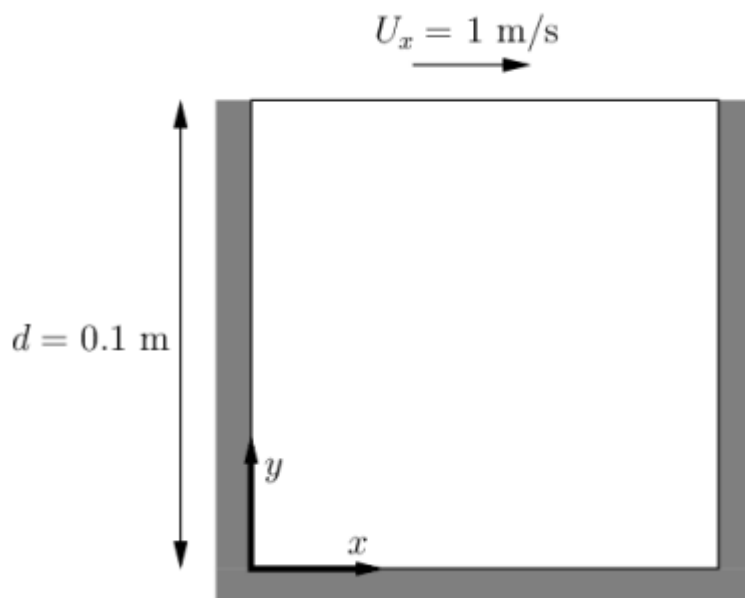


Рисунок 1 – Геометрия полости

В OpenFOAM данные представляют собой набор текстовых файлов, расположенных в определенной директории.

В разделе *FOAM_TUTORIALS/incompressible/icoFoam/cavity* есть ряд задач, в данном уроке мы рассмотрим *cavity*. Для начала необходимо скопировать каталог *cavity* в свою директорию *run*. Для этого в терминале выполните команды, представленные ниже. Для быстрого перехода в директорию *run* можете использовать команду: *run*.

Листинг 1 – Копирование файлов задачи

```
cd $FOAM_RUN
cp -r $FOAM_TUTORIALS/incompressible/icoFoam/cavity/cavity .
cd cavity
```

1.1. Генерация сетки

OpenFOAM предназначен для решения задач в 3-мерном измерении. Для решения 2D задач осуществляют построение сетки в 1 ячейку в том

направлении, в котором задача решаться не будет. Также в граничных условиях таким граням задают «empty» значение.

Ниже представлена структура блока задачи *cavity* (Рисунок 2). В плоскости XY сторона квадрата имеет размер 0.1 м. По умолчанию будем строить сетку 20x20x1.

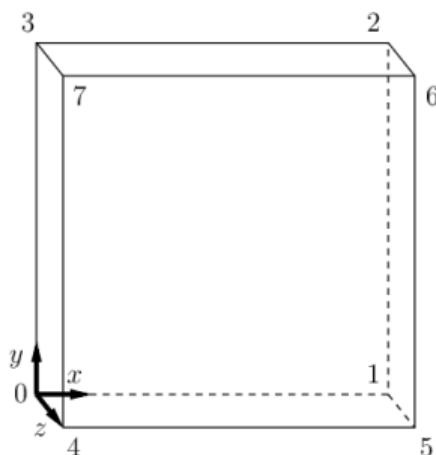


Рисунок 2 – Блочная структура сетки

Генерация сетки происходит с помощью утилиты *blockMesh*. Для построения она использует записи в файле *blockMeshDict*, который расположен в директории *system*.

Листинг 2 – Файл *blockMeshDict*

```
1/*-----*- C++ -*------*\
2 =====
3 \\\ / F ield | OpenFOAM: The Open Source CFD Toolbox
4 \\\ / O peration | Website: https://openfoam.org
5 \\\ / A nd | Version: 9
6 \\\ / M anipulation |
7\\*-----*/
8FoamFile
9{
10   format      ascii;
11   class       dictionary;
12   object      blockMeshDict;
13}
14// *****
15
16convertToMeters 0.1;
17
18vertices
19(
20   (0 0 0)
21   (1 0 0)
22   (1 1 0)
```

```

23    (0 1 0)
24    (0 0 0.1)
25    (1 0 0.1)
26    (1 1 0.1)
27    (0 1 0.1)
28);
29
30blocks
31(
32    hex (0 1 2 3 4 5 6 7) (20 20 1) simpleGrading (1 1 1)
33);
34
35edges
36(
37);
38
39boundary
40(
41    movingWall
42    {
43        type wall;
44        faces
45        (
46            (3 7 6 2)
47        );
48    }
49    fixedWalls
50    {
51        type wall;
52        faces
53        (
54            (0 4 7 3)
55            (2 6 5 1)
56            (1 5 4 0)
57        );
58    }
59    frontAndBack
60    {
61        type empty;
62        faces
63        (
64            (0 3 2 1)
65            (4 5 6 7)
66        );
67    }
68);
69
70mergePatchPairs
71(
72);
73
74// ***** //
```

В самом начале файл содержит информацию заголовка в форме баннера (строки 1-7). Такой баннер присутствует во всех файлах. Следующий блок *FoamFile* описывает формат записи, принадлежность к классу и сам объект.

В дальнейшей части руководства баннер и подраздел *FoamFile* будут опускаться (не будут включаться строки 1-16).

Файл *blockMeshDict* представляет собой словарь, в котором используются ключевые слова, описанные ниже.

Ключевое слово *convertToMeters* задает коэффициент масштабирования, на который умножаются все координаты вершин в описании сетки. Например, *convertToMeters 0.001*, означает, что все координаты умножаются на 0.001, т. е. значения, указанные в файле *blockMeshDict*, будут в мм.

Vertices (вершины блоков сетки) приведены в виде стандартного списка *x, y, z* координат точек.

Определение блоков содержится в *blocks*. Каждое определение представляет собой составную запись, состоящую из списка вершин, вектора, указывающего количество ячеек, требуемых в каждом направлении, тип и список коэффициентов расширения ячеек в каждом направлении. Коэффициент расширения позволяет сортировать или уточнять сетку в определенных направлениях.

Все линии, которые являются ребрами, по умолчанию представляют из себя прямую линию. Для того, чтобы задать изогнутое ребро, существует список *edges*. Список не является обязательным. Каждая запись для изогнутого края начинается с ключевого слова, указывающего тип кривой из перечисленных ниже:

- *arc*: дуга окружности с одной точкой интерполяции или углом + осью;
- *spline*: сплайн-кривая с использованием списка точек интерполяции;
- *polyLine*: набор линий со списком точек интерполяции;
- *BSpline*: кривая В-сплайна со списком точек интерполяции;
- *line*: прямая линия, по умолчанию не требующая спецификации ребра.

Затем за ключевым словом идут две вершины, которые образуют ребро. После этого необходимо указать точки интерполяции, через которые проходит ребро. Для *arc* требуется одно из следующих условий: одна точка интерполяции, которую будет пересекать дуга окружности; или угол и ось вращения для дуги. Для *spline*, *polyLine*, *BSpline* требуется список точек интерполяции.

Граница указана в списке *boundary*. Граница разбита на участки (области), где каждый участок в списке имеет свое имя в качестве ключевого слова, которое выбирается пользователем. Рекомендуется выбирать что-то, что удобно идентифицирует участок, например, *inlet*, *fixedWalls* и т. д.; имя используется в качестве идентификатора для задания граничных условий в соответствующих файлах. Для задания грани определяют 4 вершины. Порядок перечисления вершин должен быть следующим: начинать можно с любой вершины, но проходить грань изнутри блока следует по часовой стрелке.

Сетка может быть создана с использованием более чем 1 блока. Возникает проблема – связь между блоками. Во-первых, если грань одного блока также принадлежит другому блоку, грань блока будет образовывать набор внутренних граней ячеек в результирующей сетке. В качестве альтернативы, если пользователь захочет объединить грани блоков, которые не совсем совпадают друг с другом, он может сначала включить грани блоков в список *patches*. Каждая пара участков, грани которых должны быть объединены, затем может быть включена в дополнительный список с именем *mergePatchPairs*.

Сетка создается путем запуска утилиты *blockMesh*. Необходимо, находясь в директории *cavity*, набрать в терминале команду *blockMesh*.

Состояние работы *blockMesh* можно наблюдать в терминале. Если будут обнаружены ошибки в генерации сетки, то информация о них поступит пользователю на экран с указанием номера строки, где была вызвана ошибка.

1.2. Граничные и начальные условия

Задача настроена на запуск в момент времени $t = 0$ с, поэтому исходные данные хранятся в поддиректории 0 директории *cavity*. Поддиректория 0 содержит два файла (p – давление, U - скорость). Рассмотрим файл p .

Листинг 3 – Файл давления

```
16dimensions      [0 2 -2 0 0 0 0];
17
18internalField    uniform 0;
19
20boundaryField
21{
22    movingWall
23    {
24        type      zeroGradient;
25    }
26
27    fixedWalls
28    {
29        type      zeroGradient;
30    }
31
32    frontAndBack
33    {
34        type      empty;
35    }
36}
37
38// ***** //
```

В файле есть три основные записи:

- *dimensions*

Определяет величину размерности поля, здесь кинематическое давление, т. е. m^2s^{-2}

В таблице 1 приведены базовые единицы для системы SI, но OpenFOAM можно использовать с любой системой, главное, чтобы входные данные были корректны для выбранной системы единиц.

Таблица 1 – Базовые единицы измерения для SI

№	Параметр	SI
1	Масса	Килограмм (kg)
2	Длина	Метр (m)

3	Время	Секунда (s)
4	Температура	Кельвин (K)
5	Количество вещества	Моль (mol)
6	Сила тока	Ампер (A)
7	Интенсивность света	Кандела (candela)

- *internalField*

Данные поля, которое находится внутри расчетной области. Может быть задано однородным (*uniform*), описываемым одним значением; или неоднородным (*nonuniform*), где должны быть указаны все значения поля.

- *bondaryField*

Это словарь, содержащий набор записей, названия которых соответствуют каждому из названий участков границ, перечисленных в файле *boundary* в директории *polyMesh*. Каждая запись участка сама по себе является словарем, содержащим список записей ключевых слов. Обязательное поле *type* описывает условие участка поля. Остальные записи соответствуют типу выбранного условия поля, и обычно могут включать данные поля, задающие начальные условия на гранях участка.

Для *cavity* граница представлена стенками, разделенными на два участка: *fixedWalls* для неподвижных сторон и основания; *movingWall* для верхней грани. На стенках используется граничное условие *zeroGradient*, что означает «нормальный градиент давления равен нулю». Передняя и задняя стенки не будут принимать участия в расчетах, поэтому граничное условие для них *empty*.

В данной задаче начальные поля являются однородными и постоянными. Давление является кинематическим, абсолютное значение данного поля не несет важности, так как оно несжимаемо. В уравнениях участвуют производные, а не само давление, поэтому начальное поле задано *uniform 0*.

Поле скорости в файле *0/U* имеет размерность м/с. Внутренне поле должно выражаться однородными скалярными полями проекций скорости *uniform (0 0 0)*.

Граничное поле для скорости требует того же граничного условия (*empty*) для *frontAndBack* участков. Другие поверхности – это стены: на *fixedWalls* предполагается условие отсутствия скольжения, следовательно, условие *noSlip*. Верхняя грань движется со скоростью 1 м/с в направлении x, поэтому задается фиксированное значение *uniform (1 0 0)*.

1.3. Физические свойства

Каталог *constant* содержит в себе файлы с физическими свойствами. В данной задаче необходимо задать только кинематическую вязкость, поэтому в каталоге присутствует лишь один файл *transportProperties*. Данный параметр определяется ключевым словом *nu*. Изначально выполним расчет для числа Рейнольдса 10. Формула для его вычисления представлена ниже:

$$Re = \frac{d|U|}{\nu} \quad (1)$$

где *d* – длина, *|U|* – скорость, *ν* – кинематическая вязкость. Подставляем в формулу: *d* = 0.1 м, *|U|* = 1 м/с, *Re* = 10. Получаем *ν* = 0.01 м²с⁻¹.

1.4. Управление

В каталоге *system* находится файл *controlDict*, который отвечает за время выполнения расчета, начало расчета, шаг по времени, запись результатов. В данном примере мы собираемся запустить расчеты при *t* = 0. Необходимо настроить файл таким образом, чтобы OpenFOAM считал данные файлов из каталога 0. Для этого поле *startTime* устанавливаем в значение 0, а полю *startFrom* присваиваем значение из *startTime*.

В качестве результата необходимо достичь стационарного состояния течения потока в полости. Для ламинарного потока жидкости следует пройти 10 раз через определенную область, после этого поток становится стационарным. В задаче с полостью будем рассматривать движение верхней грани. Для прохождения пути 10 раз при скорости 0.1 м/с потребуется 1

секунда. Значение *stopAt* указываем как *endTime*, последнему задаем время 1 секунда.

Для того, чтобы решение сходилось, необходимо соблюдать правило, согласно которому число Куранта должно быть всегда меньше 1. Число Куранта можно вычислить по формуле (для одной ячейки):

$$C_0 = \frac{\delta t |U|}{\delta x} \quad (2)$$

где δt – шаг по времени, $|U|$ – скорость, δx – размер ячейки. Максимум C_0 можно наблюдать около верхней грани, где скорость приближается к 1 м/с. Размер ячейки равен:

$$\delta x = \frac{d}{n} = \frac{0.1}{20} = 0.005 \text{ м} \quad (3)$$

Поэтому для достижения числа Куранта, меньшего либо равного единице во всей области, значение временного шага *deltaT* должно быть меньше или равно:

$$\delta t = \frac{C_0 \delta x}{|U|} = \frac{1 \times 0.005}{1} = 0.005 \text{ с} \quad (4)$$

По мере выполнения расчетов будем записывать результаты через определенные промежутки времени. Ключевое слово *writeControl* предоставляет несколько вариантов настройки времени записи результатов. В данном случае выбираем параметр *timeStep*, который указывает, что результаты будут записаны на каждом *n*-ом временном шаге, где значение *n* указано с помощью ключевого слова *writeInterval*. Записывать результаты будем в 0.1, 0.2, ..., 0.5 с, с шагом по времени 0.005 с. Следовательно, необходимо выводить результаты на каждом 20-м временном шаге, поэтому *writeInterval* устанавливаем значение 20.

OpenFOAM создает новый каталог и называет его в соответствии с временем, при котором этот расчет был выполнен, например, 0.1, 0.2 и т. д.

Записи файла *controlDict* представлены ниже.

Листинг 4 – Настройка файла controlDict

```
16
17application      icoFoam;
18
19startFrom        startTime;
20
21startTime        0;
22
23stopAt           endTime;
24
25endTime          1;
26
27deltaT           0.005;
28
29writeControl      timeStep;
30
31writeInterval     20;
32
33purgeWrite        0;
34
35writeFormat       ascii;
36
37writePrecision    6;
38
39writeCompression  off;
40
41timeFormat        general;
42
43timePrecision     6;
44
45runTimeModifiable true;
46
47
48// ***** //
```

1.5. Файлы настройки решения

В файле *fvSolution* в каталоге *system* можно настроить допуски при расчете, алгоритмы управления расчетом. В файле *fvSchemes* настраиваются типы схем, которые будут использоваться при расчетах. Например, Gauss будет означать, что была выбрана центральная разностная схема, а linear говорит OpenFOAM, что интерполяция будет происходить линейно с центров ячеек на центры граней.

2 Просмотр сетки

Перед запуском выполнения расчета рекомендуется проверить сетку на наличие ошибок. Сделать это можно в ParaView (инструмент постобработки, поставляемый с OpenFOAM). Необходимо запустить скрипт paraFoam из каталога задачи.

Любой исполняемый файл UNIX/Linux может быть запущен двумя способами: как основной процесс т. е. тот, при котором командная оболочка ожидает завершения выполнения команды прежде, чем выдавать командную строку; как фоновый процесс, который позволяет командной оболочке принимать дополнительные команды, пока она еще не завершила предыдущий процесс. Поскольку удобно держать ParaView открытым во время выполнения других команд из терминала, запустим его в фоновом режиме с помощью оператора `&`, набрав: `paraFoam &`

Если текущий каталог не является *cavity*, то нужно набрать следующую команду: `paraFoam -case $FOAM_RUN/cavity &`

После выполнения данной команды откроется ParaView (Рисунок 3). На панели *Mesh Parts* можно выбрать, какую геометрию следует отобразить. Выбираем всю геометрию (все чекбоксы устанавливаем в значение true). Нажимаем на кнопку *Apply*, геометрия должна отобразиться в ParaView. Меняем на верхней панели значение *Surface* на *Surface With Edges*.

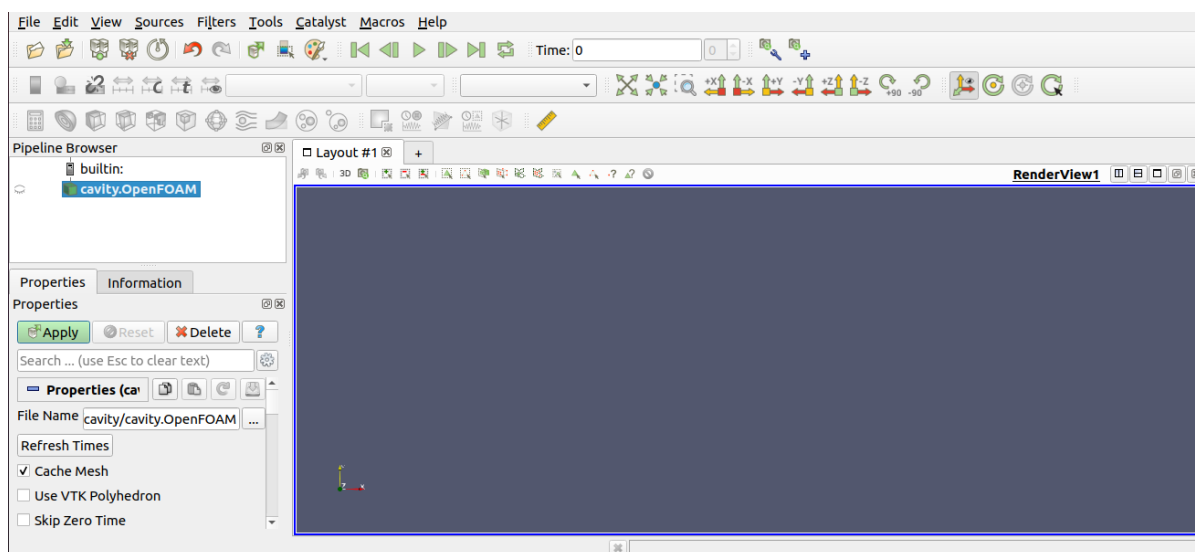


Рисунок 3 – Окно ParaView

3 Запуск приложения

В данном случае будем выполнять решение в качестве основного процесса. Решатель *isoFoam* запускается путем ввода в командной строке, находясь в нужной директории, команды *isoFoam*.

Расчет показывается в терминале. Выводится информация о текущем времени, максимальном числе Куранта, начальных и конечных расхождениях решения по всем направлениям течения.

4 Постобработка

Как только результаты будут записаны во временные каталоги, их можно будет просмотреть с помощью paraView. Вернитесь в paraView и выберите панель *Prorepties* для модуля *cavity.OpenFOAM*. Убедитесь в том, что слева от названия символ глаза находится во включенном состоянии, это необходимо для отображения графики.

Если ParaView был открыт во время выполнения расчета, то результаты по времени не будут автоматически загружены. Чтобы загрузить данные, пользователь должен убрать галочку с *Cache Mesh* и нажать на кнопку *Refresh Times* в верхней части окна *Properties* (Рисунок 4). Примените полученные изменения. Данные будут загружены в ParaView.

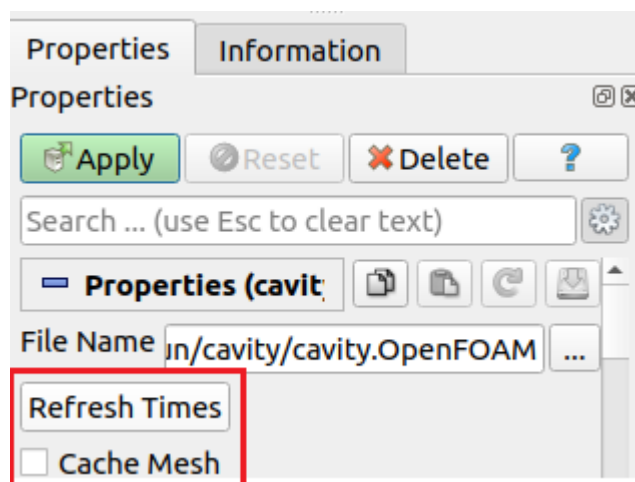


Рисунок 4 – Загрузка данных

Для вывода результата в определенное время, пользователь должен использовать *VCR Controls* или *Current Time Controls* (Рисунок 5).

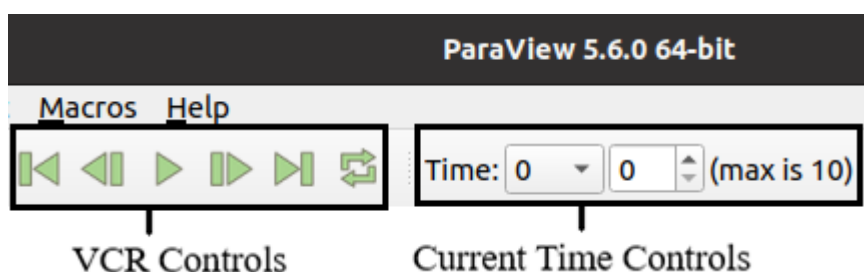


Рисунок 5 – Панель инструментов

4.1. Отображение поверхностей

Чтобы просмотреть давление, пользователь должен перейти на панель *Display*, так как она управляет визуальным представлением выбранного модуля. Чтобы построить простой график давления, пользователь должен сделать следующее (Рисунок 6):

- выбрать *Surface* в меню *Representation*;
- выбрать давление (p) в *Coloring*;
- нажать кнопку *Rescale*, чтобы установить цветовую шкалу в соответствии с диапазоном значений (при необходимости).

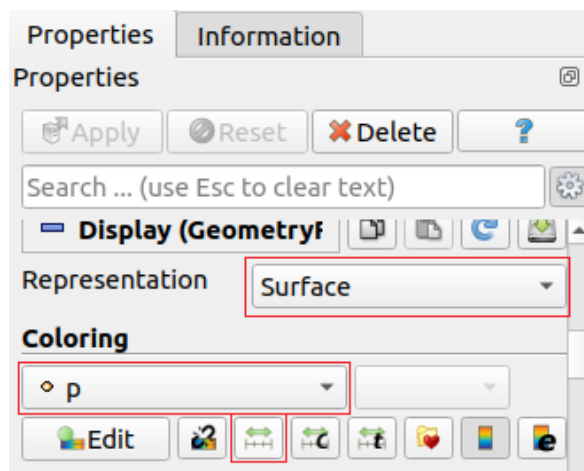


Рисунок 6 – Настройка свойств для отображения давления

Распределение давления на момент времени 1 с должно выглядеть так, как показано на рисунке 7. Область низкого давления находится в верхнем левом углу, а область высокого давления в правом верхнем углу.

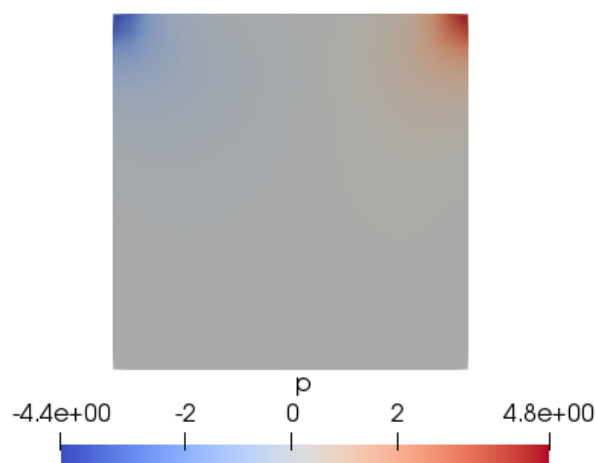


Рисунок 7 – Распределение давления в полости

С помощью значка  поле давления отображается непрерывно по всей поверхности. Если выбрать значок  в меню *Coloring*, то каждая ячейка будет иметь одно единственное значение, и не будет плавного перехода при отображении.

Цветовая легенда может быть добавлена с помощью *Toggle Color Legend Visibility*, которая расположена на *Active Variable Controls* или кнопки *Show* в разделе *Coloring* на панели *Display* (Рисунок 8).

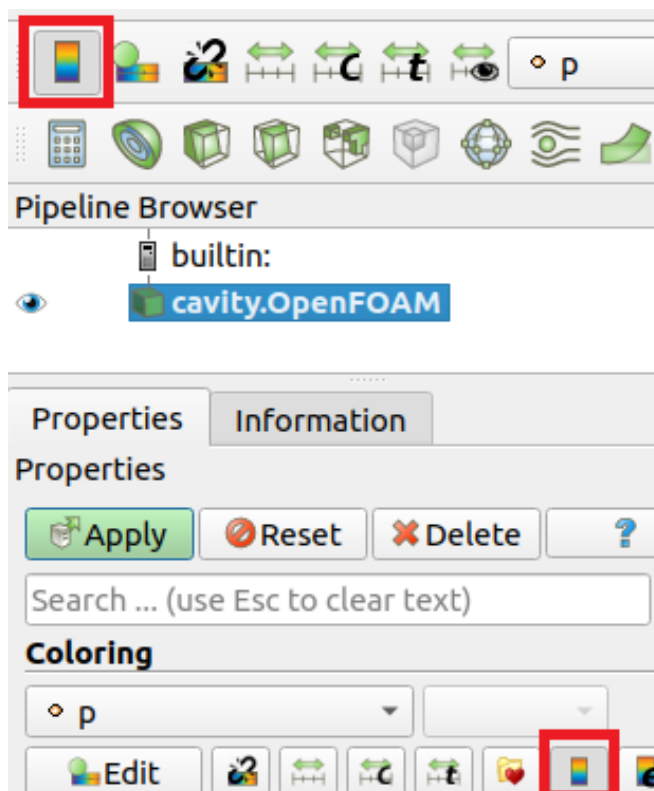


Рисунок 8 – Включение цветовой легенды

Кнопка *Edit* как на панели *Active Variable Controls*, так и на панели *Display* открывает окно *Color Map Editor*, как показано на рисунке 9, где пользователь может настроить отображение легенды, изменить шрифт и другие характеристики.

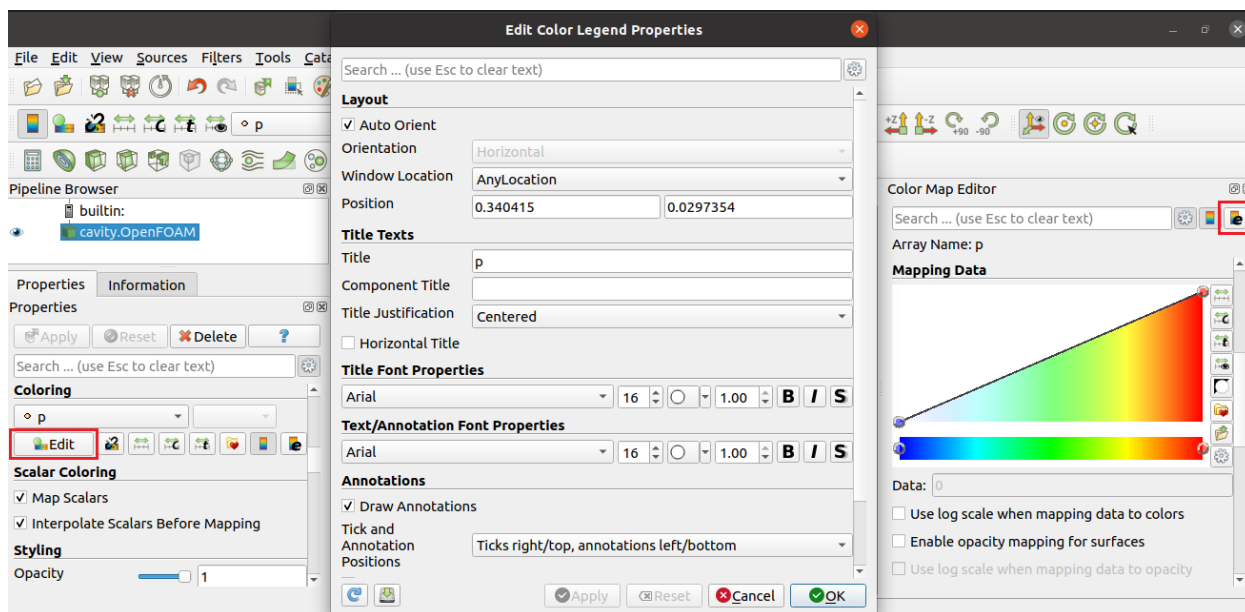


Рисунок 9 – Окно Color Map Editor

По умолчанию ParaView использует цветовую шкалу синий-белый-красный, а не более распространенную синий-зеленый-красный (радуга). Пользователь может изменить цветовую гамму, для этого необходимо нажать на кнопку  в *Color Scale Editor* и выбрать *Blue to Red Rainbow* (если такого нет, нажмите на шестеренку).

4.2. Плоскость сечения (срез)

Если вращать изображение с помощью нажатия мыши и перемещения курсора, то можно заметить, что вся геометрическая поверхность окрасилась по давлению. Для получения истинного 2-мерного контурного графика пользователь должен создать плоскость сечения (срез). С выбранным *cavity.OpenFOAM* на панели *Pipeline Browser* пользователь должен выбрать *Slice* фильтр в меню *Filters* в верхнем меню ParaView. Фильтр *Slice* находится в подменю *Common*. Плоскость сечения должна быть сосредоточена в (0.05, 0.05, 0.005), а ее нормаль должна быть установлена на (0, 0, 1), для этого нажмите кнопку *Z Normal*. Также можете выключить видимость плоскости, для этого нажмите на чекбокс *Show Plane*. Примите изменения.

4.3. Контурные

Сформировав плоскость, контуры можно получить с помощью фильтра *Contour*. С выбранным модулем *Slice* на панели *Pipeline Browser*, пользователь должен выбрать *Contour* фильтр. На панели *Properties* пользователь должен выбрать давление в меню *Contour By*. В разделе *Isosurfaces* пользователь может удалить значение по умолчанию с помощью кнопки минус, а затем добавить диапазон из 10 значений. Контурные могут отображаться в виде *Wireframes*, если *Coloring* задан как сплошной или как поле, например, давление (Рисунок 10).

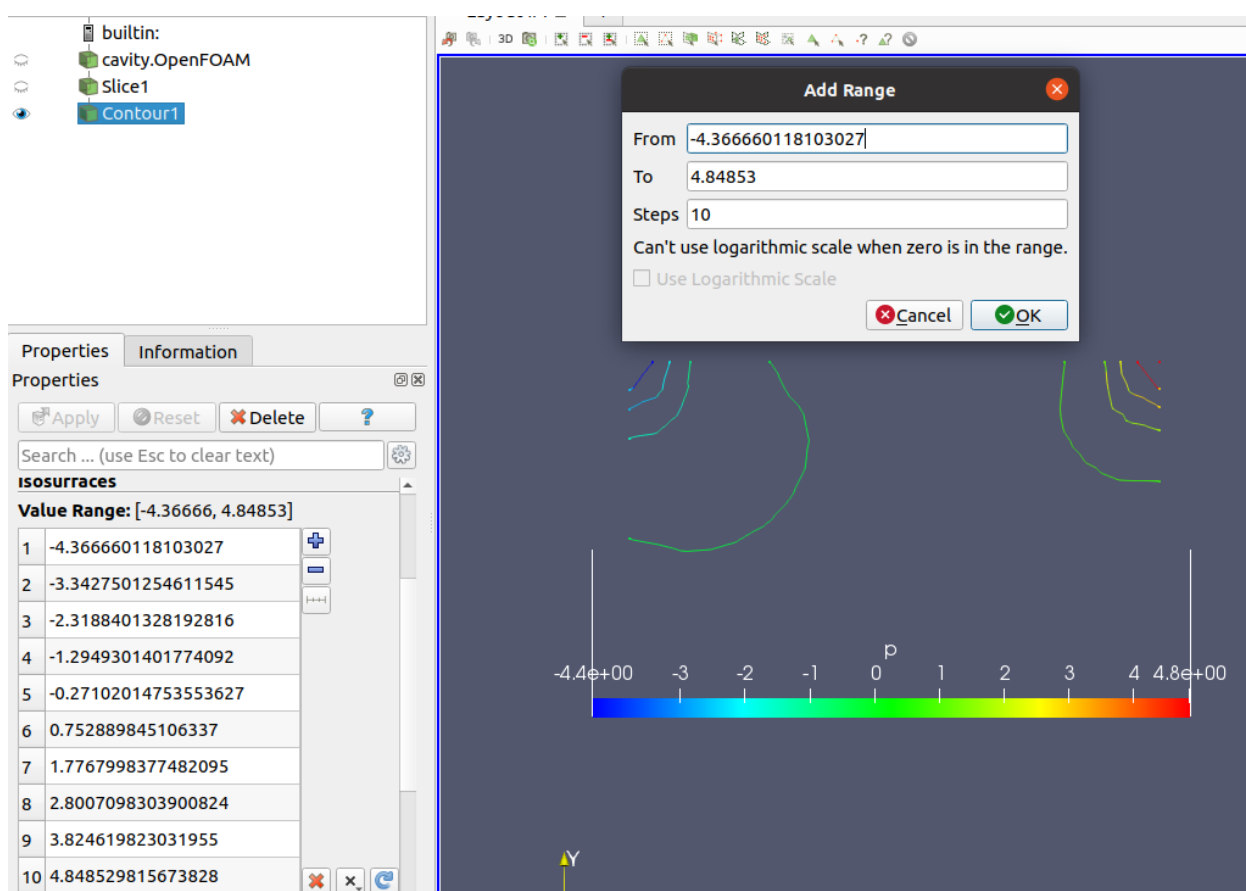


Рисунок 10 – Создание контуров

4.4. Векторные графики

Прежде чем начинать строить векторы скорости потока, не помешает удалить другие созданные модули. Их можно полностью удалить либо выключить (кнопка глаза около соответствующего модуля).

Перейдем к построению векторных скоростей по каждой ячейке. Необходимо сместить их центры, ведь по умолчанию вектора будут строиться на вершинах ячеек. Нажмите на *cavity.OpenFOAM* в *Pipeline Browser* и в меню *Filters => Alphabetical* выберите фильтр *Cell Centers*, нажмите на кнопку *Apply*. Далее к полученному модулю применим фильтр *Glyph* из меню *Filters => Common*. В панели окна *Properties* необходимо выполнить изменения: *Glyph type* поставить в значение *Arrow*, *Orientation Array* – *U*, *Scale Array* – *No scale array*, *Scale Factor* – *0.005*, *Glyph Mode* – *All Points*. После нажать кнопку *Apply*. Затем выбрать *Coloring* – *U*. Далее необходимо настроить цветовую легенду. В поле *Label Format* необходимо ввести *%-#6.2f* для установки формата вывода с двумя значащими цифрами, изначально отключив *Automatic Label Format*. Также можно изменить цветовую легенду на более привычную *rainbow*, применить фильтр *slice*, установив в его настройках *Z Normal*. Результат показан на рисунке 11.

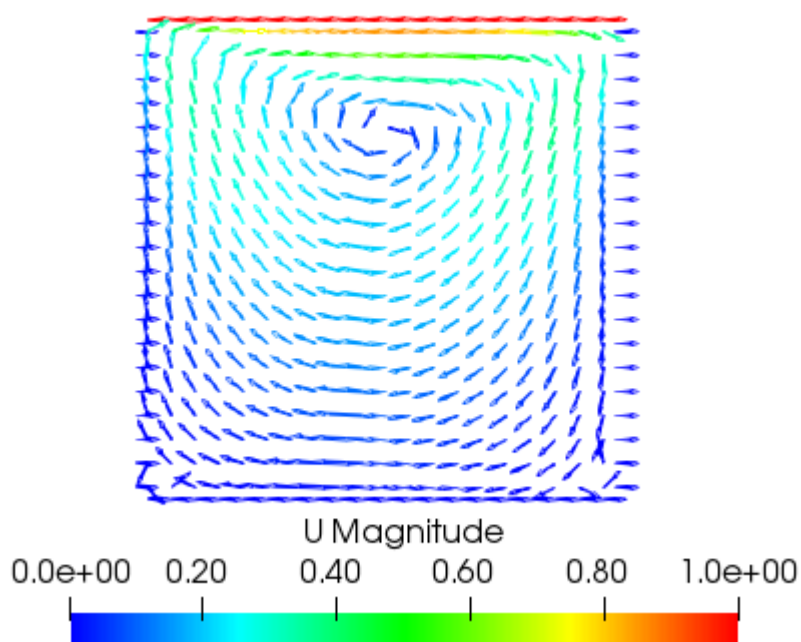


Рисунок 11 – Распределение скоростей

ParaView выбирает ориентацию символов в направлении оси *x*, когда масштабирование символов выключено, и величина скорости равна 0, именно поэтому пользователь может заметить на левой и правой границах

направление скорости показано через стенки, но на самом деле значение этой скорости равно 0.

4.5. Оптимизация участков

Прежде чем продолжать постпроцессную обработку в ParaView, необходимо отключить модули, такие как модуль векторных диаграмм, описанный выше. Теперь будем строить линии тока поля скорости.

Линии тока создаются, используя фильтр *Stream Tracer*. Панель *Seed* определяет распределение точек трассировки по *Line Source* или *Point Cloud*. Расстояние, пройденное трассировщиком, и длина шагов, которые он делает, указаны в текстовых полях на главной панели *Stream Tracer*. После создания линий фильтр *Tubes* может быть применен к модулю *Tracer* для получения изображения высокого качества. Трубки следуют за каждой линией трассировки и не являются строго цилиндрическими, но имеют фиксированное количество сторон и заданный радиус. Когда количество сторон установлено, скажем, выше 10, трубы выглядят цилиндрическими, но это увеличивает вычислительные затраты.

С выбранным модулем *cavity.OpenFOAM* в *Pipeline Browser* пользователь должен выбрать *Stream Tracer* из меню *Filter*, затем нажать на кнопку *Apply*. Убедитесь, что в окне свойств установлены следующие значения: *Integration Step Unit* – *Cell Length*, *Seed Type* – *High Resolution Line Source*. Точки *Seed* должны быть указаны в направлении *High Resolution Line Source* вертикально проходящей через центр геометрии, т. е. от (0.05, 0, 0.005) до (0.05, 0.1, 0.005). Для изображений были использованы следующие настройки: *Resolution* 21; *Maximum Step Length* 0.5; *Initial Step Length* 0.2, *Integration Direction* BOTH. *IntegratorType* Runge-Kutta 4-5.

При нажатии на кнопку *Apply* формируется индикатор (tracer). Затем пользователь должен выбрать *Tube* из меню *Filter*, чтобы получить высококачественное изображение линий тока. Проверьте свойства, которые установлены для фильтра: *Number of sides* 6, *Radius* 0.0003, *Radius factor* 10. Трубки окрашены в зависимости от величины скорости (Рисунок 12).

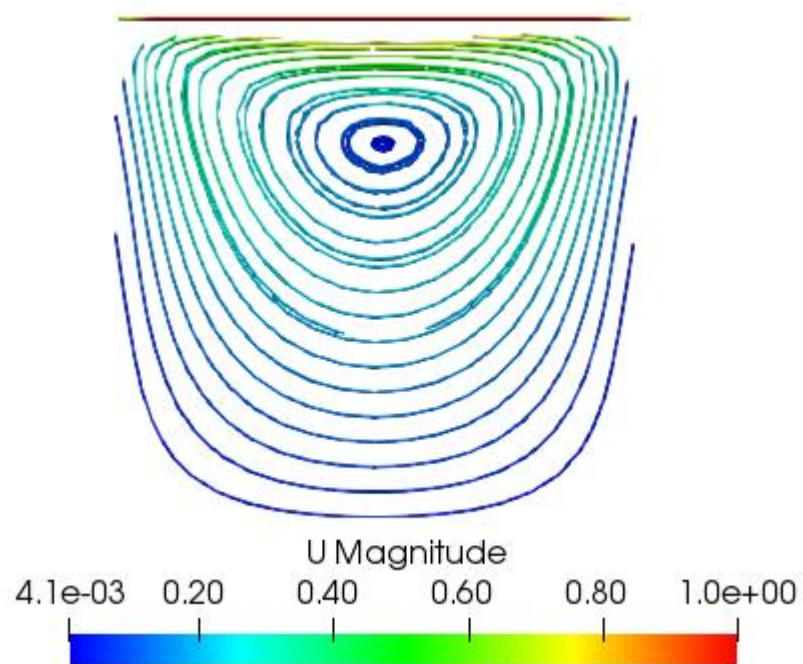


Рисунок 12 – Линии тока

5 Увеличение разрешения сетки

Разрешение сетки теперь будет увеличено в два раза в каждом направлении. Результаты грубой сетки будут перенесены на мелкую сетку в качестве начальных условий задачи. Затем сравним решение на тонкой сетке с решением на грубой сетке.

5.1. Создание нового примера на основе существующего

Теперь пользователь должен перейти в каталог *run*, набрав в терминале: `cd $FOAM_RUN`. Создадим новый пример на основе существующего.

Также существует и более удобный способ для перехода в эту директорию: просто наберите *run* в терминале.

Для начала необходимо создать новую директорию *cavityFine*, и скопировать в нее основные каталоги из *cavity*, после перейти в новый каталог.

Листинг 5 – Создание нового примера

```
mkdir cavityFine
cp -r cavity/constant cavityFine
cp -r cavity/system cavityFine
cd cavityFine
```

5.2. Создание более качественной сетки

Количество ячеек будет увеличено, для этого необходимо изменить значения в файле *blockMeshDict*. Блоки указаны в списке под ключевым словом *blocks*. Изначально идет список вершин, далее список, который задает число ячеек в каждом направлении. Первоначально это значение было равно (20 20 1) для случая *cavity*. Теперь его необходимо изменить на (40 40 1) и сохранить файл. Затем новая сетка должна быть создана путем запуска *blockMesh*.

5.3. Перенос результатов с грубой сетки на мелкую

Для переноса решения с одной модели на другую существует специальная утилита, которая называется *mapFields*. В данном примере будет использовать флаг *-consistent*, так как поля в обеих задачах являются согласованными, ведь и геометрия, и условия совпадают.

В этом примере мы хотим отобразить результаты задачи с грубой сеткой *cavity* в решение с мелкой сеткой *cavityFine*. Вы могли заметить, что решение в момент времени 1 с мало чем отличается от решения 0.5 с, это означает, что половины секунды достаточно для установления стационарного течения в полости, поэтому далее будем использовать 0.5 с. Значение *startTime* должно быть установлено 0.5, и *startFrom* должно быть равно *startTime* в словаре *controlDict* в текущей задаче.

Все готово к запуску *mapField*. Ввод *mapFields -help* показывает, что *mapFields* в качестве аргумента требует каталог примера-источника. Запускаем команду: *mapFields -consistent ../cavity*.

Листинг 6 – Вывод в терминал:

```
Source: ".." "cavity"
Target: "." "cavityFine"

Create databases as time

Source time: 0.5
Target time: 0.5

Create meshes

Source mesh size: 400   Target mesh size: 1600

Consistently creating and mapping fields for time 0.5

    interpolating p
    interpolating U

End
```

5.4. Контрольная корректировка

Для того, чтобы обеспечить сходимость решения, необходимо сохранить число Куранта меньшим 1. Для этого временной шаг необходимо уменьшить в два раза. В файле *controlDict* необходимо установить значение *deltaT* равным 0.0025 с. Данные поля сейчас записываются с интервалом равным фиксированному количеству временных шагов. Сейчас рассмотрим, как указать вывод данных через фиксированные промежутки времени. После ключевого слова *writeControl* в *controlDict* вместо запроса на вывод после

фиксированного числа временных шагов *timeStep* поставим *runTime*. В этом случае пользователь должен обеспечить вывод каждые 0.1 с, поэтому необходимо установить *writeInterval* равным 0.1, а *writeControl* установить равным *runTime*. Наконец, поскольку пример начинается с решения, полученного на грубой сетке, нам нужно запустить его только в течение короткого промежутка времени, чтобы достичь приемлемой сходимости к стационарному состоянию. Поэтому *endTime* устанавливаем на 0.7 с. Убедитесь, что эти настройки верны, затем сохраните файл. Запустите решение используя команду *icoFoam*.

5.5. Векторный график для случая с утонченной сеткой

В ParaView можно сразу открывать несколько модулей. При этом есть один нюанс в ParaView. Он ожидает, что данные примера будут храниться в одном файле с расширением, позволяющим устанавливать формат, но OpenFOAM хранит данные примера в нескольких файлах без расширения в имени, используя определенную структуру каталогов. ParaView работает так, что при открытии данных примера OpenFOAM ему передается фиктивный (пустой) файл с расширением .OpenFOAM. Скрипт *paraFoam* автоматически создает такой файл, поэтому *cavity* назывался *cavity.OpenFOAM*.

Если пользователь хочет открыть второй пример из окна ParaView, ему необходимо создать такой фиктивный файл. Это можно сделать вручную, используя скрипт *paraFoam* с опцией *-touch*. Из каталога *cavityFine* нужно выполнить: *paraFoam -touch*.

Теперь *cavityFine* можно загрузить в ParaView, выбрав созданный файл при стандартном открытии файла через меню. Теперь можно создавать векторный график.

5.6. Построение графиков

Пользователь может визуализировать результаты, извлекая некоторую скалярную величину скорости и рисуя 2-мерные графики вдоль линий, проходящих через область. Существует множество утилит, которые выполняют специализированные операции с данными, а также утилита

postProcess, которая включает в себя широкий спектр общих функций постобработки. Функции внутри *postprocess* можно просмотреть, набрав следующую команду: *postProcess -list*.

Функции *components* и *mag* предоставляют доступ к скалярным измерениям скорости. Пользователь может запустить *postprocess* с функцией *components* как на *cavity*, так и на *cavityFine*. Например, для *cavity* пользователь должен зайти в соответствующую директорию и выполнить *postProcess*, как показано ниже.

Листинг 7 – Запуск *postProcess*

```
cd $FOAM_RUN/cavity
postProcess -func "components(U)"
```

Отдельные компоненты можно отобразить в виде графика. Сгенерированные поля U_x , U_y , U_z необходимо снова загрузить в ParaView, путем нажатия на кнопку *Refresh Times*. Необходимо убедиться, что изменения применились, возможно, потребуется вновь нажать на кнопку *Apply*. Также необходимо отменить выбор *movingWall*, *fixedWall*, *frontAndBack* на панели *Mesh Parts*. Применить изменения.

Для отображения графика необходимо выбранному модулю применить фильтр *Plot Over Line (Filter => Data Analysis)*. Далее следует установить текстовые поля *Point1* и *Point2* от (0.05, 0, 0.005) до (0.05, 0.1, 0.005). *Resolution* может быть установлено равным 100.

По нажатию на кнопку *Apply* в окне XY Plot создастся график. *Attribute type* необходимо установить значение *Point Data*. Для данных по оси X можно установить *arc_length*, чтобы ось графика представляла расстояние от основания полости.

Пользователь должен отменить отображение всех серий, кроме U_x . Цветной квадрат в соседнем столбце показывает цвет линии, его можно изменить – дважды щелкните по квадрату.

Чтобы отформатировать график, пользователь может изменить настройки под разделом *Series Parameters*, такие как *Line Style*, *Chart Axes* и

другие. Также пользователь может настроить шрифт, выравнивание заголовков осей и т. д. Рисунок 13 представляет собой график, созданный с помощью ParaView.

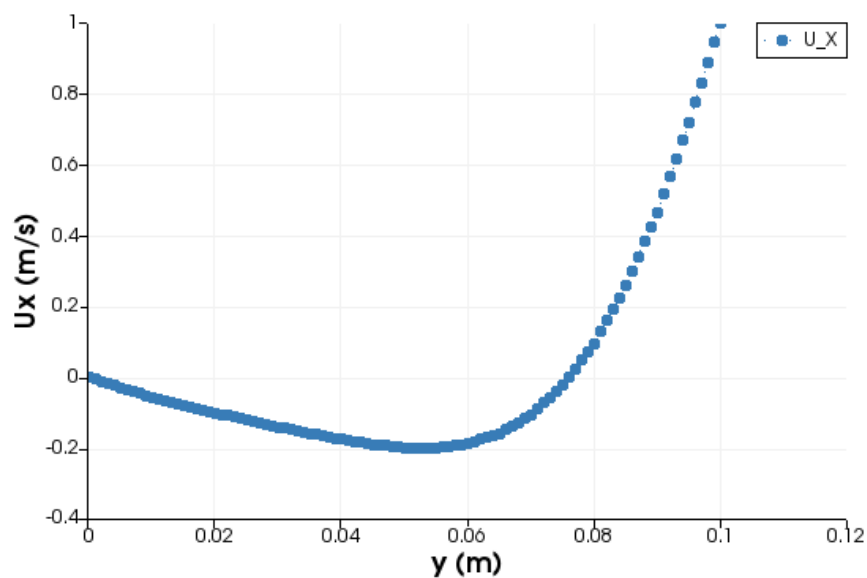


Рисунок 13 – Построение графиков в paraView

6 НАСТРОЙКА СЕТКИ

Значения областей, в которых формы истинного решения и полученного при помощи выбранных численных схем значительно отличаются, будут вычислены с наибольшей ошибкой. При уменьшении размера ячейки ошибки будут уменьшаться.

Добиться достаточной точности можно не только увеличивая количество ячеек. Хорошим решением будет сгустить сетку в направлении, где возможно получение ошибок, тем самым самые маленькие ячейки будут располагаться в этих областях, и ресурсы на вычисление будут увеличены не так значительно. В данном примере сгущение необходимо настроить по направлению к стенкам полости. Копируем в директорию *run* учебный пример *cavityGrade* из *\$FOAM_TUTORIALS/incompressible/icoFoam/cavity*.

6.1. Настройка сетки

Теперь сетка будет состоять из 4 блоков, каждый из которых имеет свою сгущенность по направлению к стенкам. Сетка из блоков показана на рисунке 14.

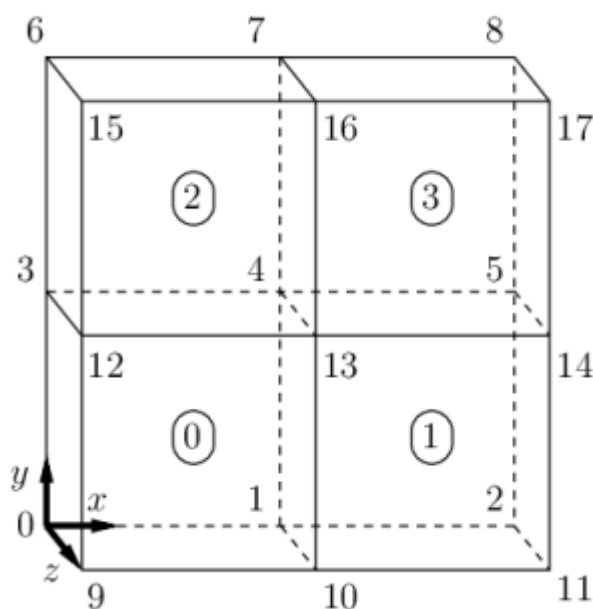


Рисунок 14 – Структура из блоков сгущенной сетки полости (номера блоков заключены в круги)

Файл *blockMeshDict* показан ниже. Каждый блок состоит из 10 ячеек двух направлений (x, y) и 1 ячейке в направлении z. Соотношение между наибольшей и наименьшей ячейками равно 2.

Листинг 8 – Файл *blockMeshDict cavityGrade*

```
16convertToMeters 0.1;
17
18vertices
19(
20    (0 0 0)
21    (0.5 0 0)
22    (1 0 0)
23    (0 0.5 0)
24    (0.5 0.5 0)
25    (1 0.5 0)
26    (0 1 0)
27    (0.5 1 0)
28    (1 1 0)
29    (0 0 0.1)
30    (0.5 0 0.1)
31    (1 0 0.1)
32    (0 0.5 0.1)
33    (0.5 0.5 0.1)
34    (1 0.5 0.1)
35    (0 1 0.1)
36    (0.5 1 0.1)
37    (1 1 0.1)
38);
39
40blocks
41(
42    hex (0 1 4 3 9 10 13 12) (10 10 1) simpleGrading (2 2 1)
43    hex (1 2 5 4 10 11 14 13) (10 10 1) simpleGrading (0.5 2 1)
44    hex (3 4 7 6 12 13 16 15) (10 10 1) simpleGrading (2 0.5 1)
45    hex (4 5 8 7 13 14 17 16) (10 10 1) simpleGrading (0.5 0.5 1)
46);
47
48edges
49(
50);
51
52boundary
53(
54    movingWall
55    {
56        type wall;
57        faces
```

```

58      (
59          (6 15 16 7)
60          (7 16 17 8)
61      );
62  }
63  fixedWalls
64  {
65      type wall;
66      faces
67      (
68          (3 12 15 6)
69          (0 9 12 3)
70          (0 1 10 9)
71          (1 2 11 10)
72          (2 5 14 11)
73          (5 8 17 14)
74      );
75  }
76  frontAndBack
77  {
78      type empty;
79      faces
80      (
81          (0 3 4 1)
82          (1 4 5 2)
83          (3 6 7 4)
84          (4 7 8 5)
85          (9 10 13 12)
86          (10 11 14 13)
87          (12 13 16 15)
88          (13 14 17 16)
89      );
90  }
91);
92
93mergePatchPairs
94(
95);
96
97// ***** //

```

В командной строке необходимо ввести команду для построения сетки с помощью утилиты *blockMesh*. Посмотреть полученную сетку можно выполнив команду для открытия ParaView – *paraFoam*.

6.2. Изменение времени расчета и временного шага

Самые большие значения скорости и самые маленькие ячейки находятся рядом с верхней гранью, поэтому наибольшее число Куранта будет рядом с ней. Поэтому следует оценить размер ячеек рядом с верхней граней, чтобы рассчитать соответствующий временной шаг.

При создании сетки со сгущением в каком-либо направлении размеры ячеек вычисляются при помощи геометрической прогрессии. При заданной длине l , количеством ячеек n и соотношением первой и последней ячейками R размер самой маленькой ячейки δx_s вычисляется по формуле:

$$\delta x_s = l \frac{r - 1}{\alpha r - 1} \quad (5)$$

где r – соотношение между размерами текущей и последующей ячейками, которое задается следующим выражением:

$$r = R^{\frac{1}{n-1}} \quad (6)$$

и

$$\alpha = \begin{cases} R & \text{для } R > 1, \\ 1 - r^{-n} + r^{-1} & \text{для } R < 1. \end{cases} \quad (7)$$

В нашем случае количество ячеек равно 10, соотношение между наибольшими и наименьшими ячейками равно 2, ширина блока и его высота равны 0.05 м. Можно вычислить наименьшую длину ячейки, которая будет равна 3.45 мм. Временной шаг следует задать меньше 3.45 мс для того, чтобы обеспечить число Куранта < 1 . Уменьшим значение δt до 2.5 мс, а $writeInterval$ зададим значение 40, для записи результатов каждые 0.1 с.

Параметр $startTime$ задаем в соответствии с условиями окончания $cavityFine$, т. е. 0.7. Поскольку $cavity$ и $cavityFine$ хорошо сходились в течение заданного времени, установим время для $cavityGrade$ 0.1 с, т. е. параметр $endTime$ необходимо задать большим или равным 0.8.

6.3. Интерполяция полей

Для переноса результатов используем утилиту *mapFields*. Из директории *cavityGrade* выполните команду: `mapFields -consistent ../cavityFine`.

Теперь из этого каталога запустим *icoFoam* и посмотрим полученные результаты, сравним их с результатами, полученными ранее. Постобработку выполняем в ParaView.

7 УВЕЛИЧЕНИЕ ЧИСЛА РЕЙНОЛЬДСА

В решенных ранее примерах число Рейнольдса равнялось 10. Это очень маленькое значение, которое быстро приводит к стабильному решению с небольшими вихрями, расположенными в нижних углах. На данном этапе увеличим число Рейнольдса до значения 100. При решении необходимо будет потратить большее время для обеспечения сходимости. Первоначально будет использоваться самая грубая сетка. Пользователь должен клонировать пример *cavity* и переименовать его в *cavityHighRe*. Вместо копирования отдельных каталогов (*system*, *constant*...), как описано было ранее, можно использовать *foamCloneCase*, который копирует соответствующие каталоги за один шаг. По умолчанию копируется каталог времени 0, но здесь пользователь может использовать опцию *-latestTime* для копирования каталога последнего времени, который можно использовать в качестве исходных данных поля для моделирования. В примере также используется быстрый переход в директорию *run*.

Листинг 9 – Клонирование каталога

```
run
foamCloneCase -latestTime cavity cavityHighRe
cd cavityHighRe
```

7.1. Пре-процессная подготовка

Перейдите в пример *cavityHighRe* и отредактируйте файл *transportProperties* в директории *constant*. Необходимо задать значение кинематической вязкости 1×10^{-3} м²/с, так как число Рейнольдса было увеличено в 10 раз. Запустим этот пример, начиная с последнего файла решения *cavity*. Для этого *startFrom* установим в значение *latestTime*. Параметр *endTime* установим равным 2 с.

7.2. Запуск кода

Запустите решатель *icoFoam* из нужной директории и обратите внимание на время выполнения. Для решения в фоновом режиме

используются команды: *nohub* (продолжает выполняться команда при выходе пользователя из системы), *nice* (задает приоритет задания).

Листинг 10 – Запуск решения

```
nohub nice -n 19 icoFoam > log &  
cat log
```

В предыдущих запусках решений вы, возможно, заметили, что *icoFoam* довольно быстро прекращает решение для скорости, но продолжает для давления намного дольше или до конца запуска. На практике, как только *icoFoam* перестает решать для U , и начальное остаточное значение p меньше, чем допуск установленный в *fvSolution* (обычно 10^{-6}), можно считать, что сходимость достигнута, решение можно останавливать, как только данные поля будут записаны в каталог времени. Например, при сходимости примера файл *log* примера *cavityHighRe* выглядит следующим образом, в котором скорость уже сошлась, а начальное остаточное значение давления мало. *No Iterations 0* указывает на то, что решение U остановилось.

Листинг 11 – Вывод решения в файл *log*

```
Time = 1.93  
  
Courant Number mean: 0.221921 max: 0.839902  
smoothSolver: Solving for Ux, Initial residual = 8.732e-06, Final residual = 8.732e-06, No Iterations 0  
smoothSolver: Solving for Uy, Initial residual = 9.89479e-06, Final residual = 9.89479e-06, No Iterations 0  
DICPCG: Solving for p, Initial residual = 3.67425e-06, Final residual = 8.6299e-07, No Iterations 4  
time step continuity errors : sum local = 6.57942e-09, global = 3.24751e-19, cumulative = -7.20035e-19  
DICPCG: Solving for p, Initial residual = 2.60841e-06, Final residual = 7.9233e-07, No Iterations 3  
time step continuity errors : sum local = 6.2604e-09, global = -5.69597e-19, cumulative = -1.28963e-18  
ExecutionTime = 0.49 s ClockTime = 1 s  
  
Time = 1.935  
  
Courant Number mean: 0.221923 max: 0.839903  
smoothSolver: Solving for Ux, Initial residual = 8.53756e-06, Final residual = 8.53756e-06, No Iterations 0  
smoothSolver: Solving for Uy, Initial residual = 9.71212e-06, Final residual = 9.71212e-06, No Iterations 0  
DICPCG: Solving for p, Initial residual = 4.02142e-06, Final residual = 9.89442e-07, No Iterations 3  
time step continuity errors : sum local = 8.14977e-09, global = 1.8868e-19, cumulative = -1.10095e-18  
DICPCG: Solving for p, Initial residual = 2.38761e-06, Final residual = 8.44401e-07, No Iterations 3  
time step continuity errors : sum local = 7.48557e-09, global = 8.11878e-19, cumulative = -2.89073e-19  
ExecutionTime = 0.49 s ClockTime = 1 s
```

8 ПОТОК С ВЫСОКИМ ЧИСЛОМ РЕЙНОЛЬДСА

Посмотрите результаты в paraView и отобразите векторы скорости. Вторичные вихри в углах увеличились в размерах. Потом пользователь может еще больше увеличить число Рейнольдса, уменьшив вязкость, а затем еще раз запустить решение. Количество вихрей увеличивается, поэтому разрешение сетки вокруг них должно увеличиваться, чтобы разрешить более сложные схемы течения. Время сходимости прямо пропорционально числу Рейнольдса, поэтому важно для обеспечения сходимости увеличивать *endTime*.

Поток переходит в турбулентный режим. Для решения будут использоваться усредненные по Рейнольдсу (RAS) модели турбулентности. Будем использовать решатель *pisoFoam*.

8.1. Пре-процессная подготовка

Вернитесь в каталог *run* и скопируйте пример *cavity* из директории *\$FOAM_RUN/tutorials/incompressible/pisoFoam/RAS*, переименуйте его в *cavityRAS*, перейдите в новый каталог.

Листинг 12 – Копирование файла решения

```
run
cp -r $FOAM_TUTORIALS/incompressible/pisoFoam/RAS/cavity cavityRAS
cd cavityRAS
```

Создайте сетку с помощью утилиты *blockMesh*. Градуировка сетки по направлению к стене не обязательна, когда используется стандарт $k - \epsilon$ модели с функциями стены, поскольку поток в ячейке у стены модулируется, и не обязательно должен быть разрешен.

В OpenFOAM доступен ряд функциональных моделей стен, которые применяются в качестве граничных условий на отдельных участках. Это позволяет применять различные функциональные модели стен к различным областям стен. Выбор функциональных моделей стен задается через поле турбулентной вязкости ν_t в файле *0/nut*.

Листинг 13 – Файл nut

```

16
17dimensions      [0 2 -1 0 0 0 0];
18
19internalField    uniform 0;
20
21boundaryField
22{
23    movingWall
24    {
25        type      nutkWallFunction;
26        value      uniform 0;
27    }
28    fixedWalls
29    {
30        type      nutkWallFunction;
31        value      uniform 0;
32    }
33    frontAndBack
34    {
35        type      empty;
36    }
37}
38
39
40// ***** //

```

В этом случае используются стандартные функции стены, обозначенные типом *nutkWallFunction* на границах *movingWall* и *fixedWall*. Другие функциональные модели стен, включающие функции грубой сетки, указываются с помощью ключевого слова *nutRoughWallFunction*.

Теперь пользователь должен открыть файлы *k* и *ε* (*0/k*, *0/epsilon*). Тип граничных условий в файле *ε* – *epsilonWallFunction*, в файле *k* используется *kqRWallFunction*. Начальные значения для *k* и *ε* устанавливаются с использованием расчетной флуктуирующей составляющей скорости *U'* и шкалы длины турбулентности, *l*, *k* и *ε* определяются как:

$$k = \frac{1}{2} \overline{U' \cdot U'} \quad (8)$$

$$\varepsilon = \frac{C_\mu^{0.75} k^{1.5}}{l} \quad (9)$$

где C_μ – константа $k - \varepsilon$ модели, равна 0.09.

Для декартовой системы координат k :

$$k = \frac{1}{2}(U_x'^2 + U_y'^2 + U_z'^2) \quad (10)$$

где $U_x'^2$, $U_y'^2$, $U_z'^2$ – флуктуирующие компоненты скорости по всем направлениям. При изотропной начальной турбулентности и равной 5% от скорости движения верхней грани, l равна 5% ширины корпуса, 0.1 м, k и $\mathcal{E}$ задаются:

$$U_x' = U_y' = U_z' = \frac{5}{100} 1 \text{ м/с} \quad (11)$$

$$\Rightarrow k = \frac{3}{2} \left(\frac{5}{100} \right)^2 \text{ м}^2 \text{ с}^{-2} = 3.75 \times 10^{-3} \text{ м}^2 \text{ с}^{-2} \quad (12)$$

$$\varepsilon = \frac{C_\mu^{0.75} k^{1.5}}{l} \approx 7.54 \times 10^{-3} \text{ м}^2 \text{ с}^{-3} \quad (13)$$

Так задаются начальные условия для k и $\mathcal{E}$. Начальные условия для скорости и давления не изменились. Они равны $(0, 0, 0)$ и 0 соответственно.

Методы моделирования турбулентности могут быть: laminar, RAS, LES (моделирование больших вихрей). Выбор осуществляется с помощью ключевого слова *simulationType* в файле *momentumTransport*.

Листинг 14 – Файл *momentumTransport*

```
16
17simulationType  RAS;
18
19RAS
20{
21    model          kEpsilon;
22
23    turbulence      on;
24
25    printCoeffs     on;
26}
27
28// ***** //
```

В данном случае выбирается *RAS*, выбор моделирования располагается в подразделе *RAS*. Модель турбулентности задается с помощью ключевого слова *model*. Следует выбрать модель *kEpsilon*, которая является стандартной

k — $\mathcal{E}$ моделью. Пользователь также должен убедиться, что расчет *turbulence* включен *on*.

Установка *printCoeffs* в значение *on* будет означать, что значения по умолчанию будут выводиться в терминал после запуска модели.

Необходимо изменить значение ламинарной кинематической вязкости в файле *transportProperties*. Для того, чтобы число Рейнольдса достигло значения 10^4 , вязкость должна быть 10^{-5} м.

Теперь требуется внести изменения в файл *controlDict*. Значение *deltaT* должно быть равным 0.005 с, *endTime* – 10 с.

8.2. Запуск кода

Запустите решатель *pisoFoam* в терминале для начала решения (команда для запуска соответствует названию решателя). Проанализируйте значение Куранта. При числе значительно меньшем 1 будет тратиться большее время для решения. Поэтому необходимо изменить временной шаг на значение, при котором число Куранта будет наиболее близко к 1, но не будет его превышать. Поэтому измените значение *deltaT* на 0.02 с и установите *startFrom* в значение *latestTime*. Это укажет решателю *pisoFoam* считывать начальные данные из каталога последнего времени. Значение *endTime* должно быть равным 20 с, так как сходимость будет намного медленнее, чем при ламинарном течении. Перезапустите решение и следите за сходимостью. Просматривайте результаты на последовательных временных этапах по мере выполнения решения, чтобы увидеть, переходит ли решение в устойчивое состояние или, возможно, достигает периодически колеблющегося состояния.

9 ИЗМЕНЕНИЕ ГЕОМЕТРИИ КОРПУСА

В качестве примера скопируем пример *cavityClipped* из директории *tutorials* в пользовательскую директорию и перейдем в каталог *cavityClipped*.

Листинг 15 – Копирование файла решения

```
run
cp -r $FOAM_TUTORIALS/incompressible/icoFoam/cavity/cavityClipped .
cd cavityClipped
```

Корпус состоит из стандартной геометрии *cavity*, но со стороной квадрата 0.04 м, удаленным из нижнего правого угла полости.

Листинг 16 – файл *blockMeshDict*

```
16convertToMeters 0.1;
17
18vertices
19(
20     (0 0 0)
21     (0.6 0 0)
22     (0 0.4 0)
23     (0.6 0.4 0)
24     (1 0.4 0)
25     (0 1 0)
26     (0.6 1 0)
27     (1 1 0)
28
29     (0 0 0.1)
30     (0.6 0 0.1)
31     (0 0.4 0.1)
32     (0.6 0.4 0.1)
33     (1 0.4 0.1)
34     (0 1 0.1)
35     (0.6 1 0.1)
36     (1 1 0.1)
37
38);
39
40blocks
41(
42     hex (0 1 3 2 8 9 11 10) (12 8 1) simpleGrading (1 1 1)
43     hex (2 3 6 5 10 11 14 13) (12 12 1) simpleGrading (1 1 1)
44     hex (3 4 7 6 11 12 15 14) (8 12 1) simpleGrading (1 1 1)
45);
46
47edges
48(
49);
```

```

50
51boundary
52(
53    lid
54    {
55        type wall;
56        faces
57        (
58            (5 13 14 6)
59            (6 14 15 7)
60        );
61    }
62    fixedWalls
63    {
64        type wall;
65        faces
66        (
67            (0 8 10 2)
68            (2 10 13 5)
69            (7 15 12 4)
70            (4 12 11 3)
71            (3 11 9 1)
72            (1 9 8 0)
73        );
74    }
75    frontAndBack
76    {
77        type empty;
78        faces
79        (
80            (0 2 3 1)
81            (2 5 6 3)
82            (3 6 7 4)
83            (8 9 11 10)
84            (10 11 14 13)
85            (11 12 15 14)
86        );
87    }
88);
89
90mergePatchPairs
91(
92);
93
94// ***** //

```

Создайте сетку с помощью утилиты *blockMesh*. Грани устанавливаются аналогично, как и в предыдущих случаях. Для ясности описания процесса переименован участок верхней грани, который осуществляет движение. В данном случае он называется *lid* (ранее *movingWall*).

При несогласующемся сопоставлении нет гарантий, что все данные поля могут быть отображены из исходного примера. Остальные данные должны быть получены из файлов в заданном примере. Поэтому данные полей должны быть в каталоге времени решаемого примера до того, как будет выполнено сопоставление. В *cavityClipped* сопоставление должно наступить в 0.5 с, так как в файле *controlDict* значение *startTime* равно 0.5 с. Поэтому пользователю необходимо скопировать начальные данные поля в этот каталог, например, с момента времени 0, путем ввода команды в терминал: `cp -r 0 0.5`

Перед сопоставлением данных необходимо просмотреть геометрию и поля на момент времени 0.5 с.

Теперь будем отображать поля давления и скорости с прошлого решения на новые поля *cavityClipped*. Сопоставление является противоречивым, поэтому необходимо вносить изменения в файле *mapFieldsDict* из каталога *system*. Ключевые слова в файле: *patchMap* и *cuttingPatches*. Список *patchMap* показывает, какое поле от какого поля наследует значения. В нашем случае участок *lid* от *movingWall*.

Листинг 17 – Установка значений в *patchMap*

```
patchMap
(
    lid movingWall
);
```

Список *cuttingPatches* содержит участки нового решения, значения которых должны быть сопоставлены с исходным внутренним полем, где удаляются целевые грани. В данном случае участок *fixedWalls* имеет условие *noSlip*, поэтому внутренние значения не могут быть интерполированы в участок. Поэтому список может остаться пустым.

Листинг 18 – Значения в *cuttingPatches*

```
cuttingPatches
(
);
```

Если пользователь хочет интерполировать внутренние значения из исходного случая в грань *fixedWalls* целевого случая, граничное условие *fixedValue* должно быть указано на участке, у которого *value* затем может быть переопределено в процессе сопоставления. Участок *fixedWalls* должен быть включен в список *cuttingPatches*.

Пользователь должен запустить *mapFields* из директории *cavityClipped*, путем ввода команды в терминал: *mapFields ../cavity*

Пользователь может просмотреть сопоставленное поле, как показано на рисунке 15. Грань *fixedWalls* не унаследовала значения из исходного примера, как и ожидалось. Затем пользователь может запустить решение с помощью утилиты *icoFoam*.

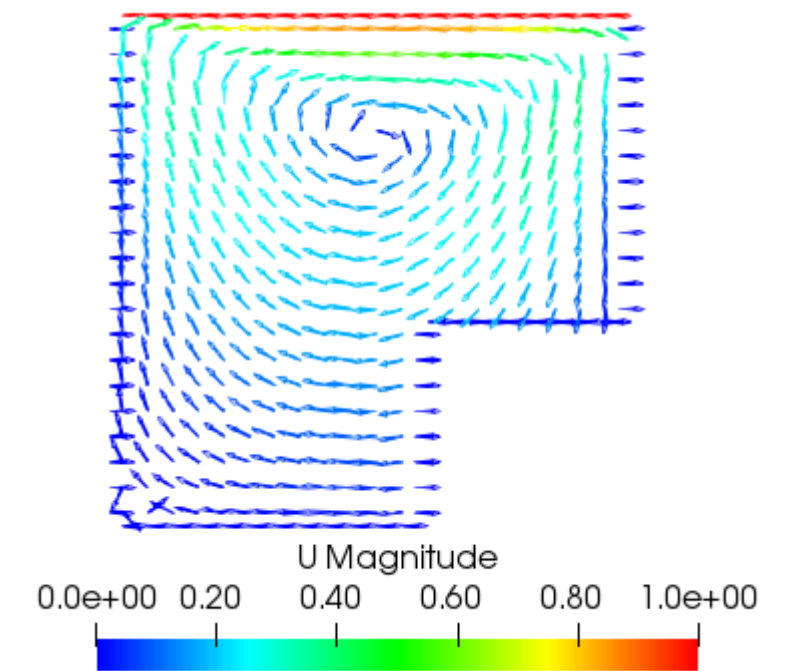


Рисунок 15 – Решение *cavity*, отображенное на *cavityClipped*

10 Постобработка измененной геометрии

Отображение скорости может быть сгенерировано как обычно, сначала для времени 0.5 с, потом для 0.6 с для последующего сравнения полученных результатов. Теперь применим фильтр для постобработки результата. Из меню *Filters* выбираем *Extract Block*, а на панели *Parameter* выбираем участки для отображения, а именно *Lid* и *fixedWalls*. Отображаем результат нажатием на кнопку *Apply*, выбираем *WireFrames* на панели *Display*. Рисунок 16 показывает полученный результат.

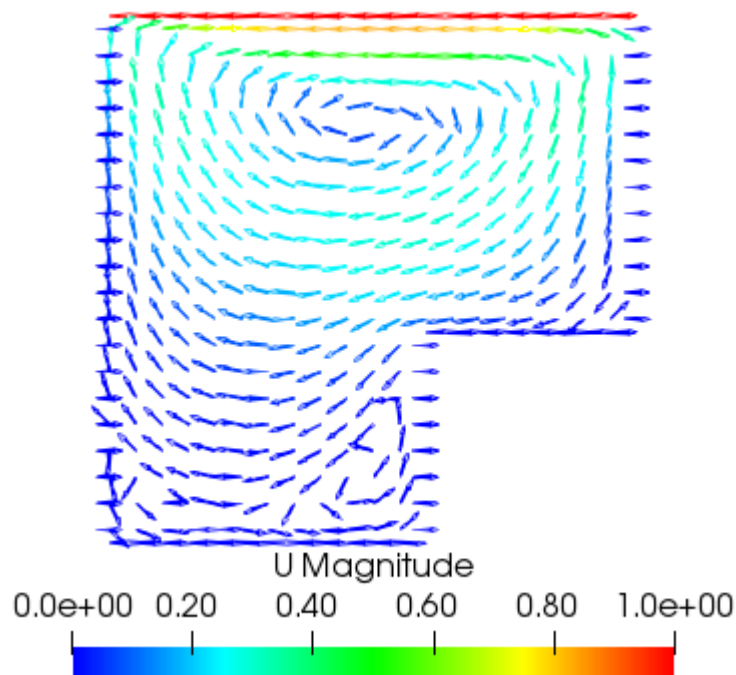


Рисунок 16 – Решение cavityClipped для поля скоростей

Заключение

В ходе данной работы был рассмотрен учебный пример из документации OpenFOAM, посвященный изотермическому ламинарному и турбулентному течению несжимаемой жидкости в двумерной квадратной области. Были выполнены этапы пре-процессинга, расчета и постобработки.

Вопросы для самоконтроля:

1. В чем заключается основное отличие решателя `icoFoam` от решателя `pisoFoam`?
2. Для чего используется утилита `mapFields`?
3. Как выполнить решение 2D задачи в OpenFOAM?
4. Какое правило необходимо соблюдать для того, чтобы обеспечить сходимость решения?
5. С помощью какой утилиты можно построить график в OpenFOAM?